Preface

The Python programming language has unique strengths and charms that can be hard to grasp. Many programmers familiar with other languages often approach Python from a limited mindset instead of embracing its full expressivity. Some programmers go too far in the other direction, overusing Python features that can cause big problems later.

This book provides insight into the *Pythonic* way of writing programs: the best way to use Python. It builds on a fundamental understanding of the language that I assume you already have. Novice programmers will learn the best practices of Python's capabilities. Experienced programmers will learn how to embrace the strangeness of a new tool with confidence.

My goal is to prepare you to make a big impact with Python.

WHAT THIS BOOK COVERS

Each chapter in this book contains a broad but related set of items. Feel free to jump between items and follow your interest. Each item contains concise and specific guidance explaining how you can write Python programs more effectively. Items include advice on what to do, what to avoid, how to strike the right balance, and why this is the best choice. Items reference each other to make it easier to fill in the gaps as you read.

This second edition of this book is focused exclusively on Python 3 (see Item 1: "Know Which Version of Python You're Using"), up to and including version 3.8. Most of the original items from the first edition have been revised and included, but many have undergone substantial updates. For some items, my advice has completely changed between the two editions of the book due to best practices evolving as Python has matured. If you're still primarily using Python 2, despite its end-of-life on January 1st, 2020, the previous edition of the book may be more useful to you.

Python takes a "batteries included" approach to its standard library, in comparison to many other languages that ship with a small number of common packages and require you to look elsewhere for important functionality. Many of these built-in packages are so closely intertwined with idiomatic Python that they may as well be part of the language specification. The full set of standard modules is too large to cover in this book, but I've included the ones that I feel are critical to be aware of and use.

Chapter 1: Pythonic Thinking

The Python community has come to use the adjective *Pythonic* to describe code that follows a particular

style. The idioms of Python have emerged over time through experience using the language and working with others. This chapter covers the best way to do the most common things in Python.

Chapter 2: Lists and Dictionaries

In Python, the most common way to organize information is in a sequence of values stored in a `list`. A list's natural complement is the dict that stores lookup keys mapped to corresponding values. This chapter covers how to build programs with these versatile building blocks.

Chapter 3: Functions

Functions in Python have a variety of extra features that make a programmer's life easier. Some are similar to capabilities in other programming languages, but many are unique to Python. This chapter covers how to use functions to clarify intention, promote reuse, and reduce bugs.

Chapter 4: Comprehensions and Generators

Python has special syntax for quickly iterating through lists, dictionaries, and sets to generate derivative data structures. It also allows for a stream of iterable values to be incrementally returned by a function. This chapter covers how these features can provide better performance, reduced memory usage, and improved readability.

Chapter 5: Classes and Interfaces

Python is an object-oriented language. Getting things done in Python often requires writing new classes and defining how they interact through their interfaces and hierarchies. This chapter covers how to use classes to express your intended behaviors with objects.

Chapter 6: Metaclasses and Attributes

Metaclasses and dynamic attributes are powerful Python features. However, they also enable you to implement extremely bizarre and unexpected behaviors. This chapter covers the common idioms for using these mechanisms to ensure that you follow the *rule of least surprise*.

Chapter 7: Concurrency and Parallelism

Python makes it easy to write concurrent programs that do many different things seemingly at the same time. Python can also be used to do parallel work through system calls, subprocesses, and C extensions. This chapter covers how to best utilize Python in these subtly different situations.

Chapter 8: Robustness and Performance

Python has built-in features and modules that aid in hardening your programs so they are dependable. Python also includes tools to help you achieve higher performance with minimal effort. This chapter covers how to use Python to optimize your programs to maximize their reliability and efficiency in production.

Chapter 9: Testing and Debugging

You should always test your code, regardless of what language it's written in. However, Python's dynamic features can increase the risk of runtime errors in unique ways. Luckily, they also make it easier to write tests and diagnose malfunctioning programs. This chapter covers Python's built-in tools for testing and debugging.

Chapter 10: Collaboration

Collaborating on Python programs requires you to be deliberate about how you write your code. Even if you're working alone, you'll want to understand how to use modules written by others. This chapter covers the standard tools and best practices that enable people to work together on Python programs.

CONVENTIONS USED IN THIS BOOK

Python code snippets in this book are `in monospace font` and have syntax highlighting. When lines are long, I use characters to show when they wrap. I truncate some snippets with ellipses (`...`) to indicate regions where code exists that isn't essential for expressing the point. You'll need to download the full example code (see below on where to get it) to get these truncated snippets to run correctly on your computer.

I take some artistic license with the Python style guide in order to make the code examples better fit the format of a book, or to highlight the most important parts. I've also left out embedded documentation to reduce the size of code examples. I strongly suggest that you don't emulate this in your projects; instead, you should follow the style guide (see Item 2: "Follow the PEP 8 Style Guide") and write documentation (see Item 84: "Write Docstrings for Every Function, Class, and Module").

Most code snippets in this book are accompanied by the corresponding output from running the code. When I say "output," I mean console or terminal output: what you see when running the Python program in an interactive interpreter. Output sections are in `monospace font` and are preceded by a `>>>` line (the Python interactive prompt). The idea is that you could type the code snippets into a Python shell and reproduce the expected output.

Finally, there are some other sections in `monospace font` that are not preceded by a `>>>` line. These represent the output of running programs besides the normal Python interpreter. These examples often

begin with `$` characters to indicate that I'm running programs from a command-line shell like Bash. If you're running these commands on Windows or another type of system, you may need to adjust the program names and arguments accordingly.

Where to Get the Code and Errata

It's useful to view some of the examples in this book as whole programs without interleaved prose. This also gives you a chance to tinker with the code yourself and understand why the program works as described. You can find the source code for all code snippets in this book on the book's website at https://effectivepython.com. The website also includes any corrections to the book, as well as how to report errors.